

Reporte Técnico del Sistema IoT con MQTT

Proyecto: Sistema de monitoreo y control IoT mediante MQTT

Equipo: Equipo 7

Broker: BeagleBone Black con Mosquitto

Publisher: ESP32 con 7 sensores reed switch

Subscriber: Dashboard web en Python

Fecha de validación: 20 de junio de 2026

1. Introducción

En este proyecto se construyó un sistema IoT para vigilar la apertura de siete compartimentos. El sistema usa el protocolo MQTT para que cada parte cumpla una tarea clara. El ESP32 toma los datos de los sensores, la BeagleBone Black recibe y reparte los mensajes, y el dashboard web muestra el estado en tiempo real y activa una alarma digital cuando detecta aperturas.

2. Objetivo

Diseñar e implementar una arquitectura IoT simple, ordenada y funcional, donde un nodo publicador, un broker MQTT y un nodo suscriptor trabajen juntos en

tiempo real. El sistema debía permitir lectura de sensores, envío de eventos, visualización web y reacción del actuador digital.

3. Descripción general del sistema

La solución final quedó dividida en tres partes:

- **ESP32:** lee siete sensores reed switch y publica el estado de cada compartimento.
- **BeagleBone Black:** ejecuta el broker Mosquitto y escucha en el puerto 1883.
- **Dashboard web:** se suscribe a los tópicos MQTT, muestra el estado y activa una alarma digital.

4. Diagrama de arquitectura del sistema

Diagrama en formato Mermaid.

```
flowchart LR
```

```
S1[Reed 1] --> ESP[ESP32 Publisher]
```

```
S2[Reed 2] --> ESP
```

```
S3[Reed 3] --> ESP
```

```
S4[Reed 4] --> ESP
```

```
S5[Reed 5] --> ESP
```

S6[Reed 6] --> ESP

S7[Reed 7] --> ESP

ESP -->|Publica datos MQTT| BBB[BeagleBone Black\nMosquitto Broker]

BBB -->|Entrega mensajes MQTT| WEB[Dashboard Web\nSubscriber]

WEB -->|Comandos MQTT| BBB

BBB -->|request-sync / set-monitoring| ESP

5. Topología de red

Durante la prueba en laboratorio se usó una red local donde cada nodo tuvo una dirección IP definida. Esta topología permitió validar la comunicación completa entre el publicador, el broker y el dashboard.

Dis
po
siti
vo

Rol

IP
o
b
se
rv
a
d

M
e
d
i

		a	o
		19	
ES	Publ	2.	W
P3	ishe	16	i
2	r	8.	F
		1.	i
		74	
			E
Be		19	t
agl	Brok	2.	h
eB	er	16	e
one	Mos	8.	r
Bla	quitt	1.	n
ck	o	85	e
			t
Lap	Das	19	W
top	hbo	2.	i
de	ard /	16	F
pru	mon	8.	i
eba	itore	1.	

Diagrama en formato Mermaid.

```
flowchart TD
```

```
NET[Red local 192.168.1.0/24]
```

```
ESP[ESP32\n192.168.1.74]
```

```
BBB[BBB\n192.168.1.85]
```

```
PC[Laptop\n192.168.1.75]
```

```
ESP --> NET
```

```
BBB --> NET
```

PC --> NET

ESP -->|MQTT 1883| BBB

PC -->|HTTP 8080| PC

PC -->|MQTT subscribe| BBB

6. Árbol de tópicos MQTT

Se definió una estructura clara y jerárquica de tópicos para separar estados, comandos y actuadores. La raíz usada en el sistema es **escom/iot/equipo7/reed-monitor**.

escom/iot/equipo7/reed-monitor/

└─ publisher/status

└─ sensor/summary

└─ sensor/heartbeat

└─ sensor/compartment/1

└─ sensor/compartment/2

└─ sensor/compartment/3

└─ sensor/compartment/4

└─ sensor/compartment/5

└─ sensor/compartment/6

└─ sensor/compartment/7

└─ command/request-sync

└─ command/set-monitoring

└─ actuator/alarm

└─ subscriber/status

7. Diagrama del flujo MQTT

Diagrama en formato Mermaid.

```
sequenceDiagram
```

```
participant ESP as ESP32
```

```
participant BBB as Broker MQTT
```

```
participant WEB as Dashboard
```

```
ESP->>BBB: publisher/status
```

```
ESP->>BBB: sensor/compartment/N
```

```
ESP->>BBB: sensor/summary
```

```
ESP->>BBB: sensor/heartbeat
```


BBB->>WEB: entrega mensajes

WEB->>BBB: command/request-sync

WEB->>BBB: command/set-monitoring

BBB->>ESP: reenvía comandos

8. Calidad de servicio usada

Tópico	QoS	Retenido	Motivo
publisher/status	1	Si	Permite saber si el publisher sigue en

				línea.
sub scri ber/ stat us	1	S í		Permite saber si el dashboard sigue en línea.
sen sor/ sum mar y	1	N o		Evita perder el estado general del sistema.
sen sor/ com part men t/#	1	N o		Ayuda a registrar cambios por compartim ento.

			S	
			e	
			g	
			ú	
			n	Asegura
com				que el
man	1		e	comando
d/#			l	llegue al
				publicador.
			c	
			a	
			s	
			o	
actu				
ator				Conserva
/	1		S	el último
alar			í	estado de
m				la alarma.

9. Desarrollo e implementación

9.1 Publisher

El ESP32 se programó en C++ con PlatformIO y el framework Arduino. El nodo lee siete sensores reed switch, detecta cambios de apertura y cierre, y publica tres tipos de mensajes: estado por compartimento, resumen general y heartbeat.

9.2 Broker

El broker se ejecutó en la BeagleBone Black con Mosquitto. Durante la

configuración se detectó que la imagen Debian 7 de la BBB era antigua, por lo que fue necesario ajustar la compatibilidad del proyecto para trabajar con MQTT 3.1.

9.3 Subscriber

El subscribir se desarrolló como una aplicación web en Python. La interfaz muestra el número de compartimentos abiertos, el estado del publisher, la alarma activa y una lista de eventos recientes.

10. Pruebas de conectividad y análisis de resultados

Se hicieron pruebas de red, de broker y de intercambio MQTT entre los nodos. Las pruebas ayudaron a detectar un problema de compatibilidad con la versión del broker de la BBB. Después del ajuste, la comunicación quedó estable.

```
graph LR; Prueba --> Comando; Comando --> o1[o]; Comando --> e1[e]; Comando --> evidencia; evidencia --> d[evidencia]; evidencia --> e2[e]; evidencia --> ResultadoEsperado; ResultadoEsperado --> o2[o]; ResultadoEsperado --> e3[e]; ResultadoEsperado --> ResultadoObservado;
```

Broker
activo

netstat -ltn | grep :1883

Mostró escuchando en el puerto 1883

Correcto. La BBB mostró escucha en 0.0.0.0:1883 y :::1883.

Conexión
activa

Prueba

Conexión
TCP
existente

Correcto. La laptop abrió conexión al broker en 192.168.1

d T C P d es d e la pt o p	d e s o c k e t a l p u e r t o 1 8 8 3	tos a Ve r me ns aje s del pu blic	.85:1883. Correcto. Se observaron mensajes en publisher /status, sensor/su mmary, sensor/co mpartment
S us cri pc ió n M Q T	M o n i t o r d		

	e l		
T e n la la pt o p	t ó p i c o r a í z	ad or	/# y sensor/he artbeat.

P u bli ca ci ón d el E S P 3 2	M o n i t o r s e r i e	Me ns aje s en via do s co n éxi to	Correcto. El monitor serie mostró publicacione s => OK.
---	--	--	---

Vi
su
ali
za
ci
ón
e
n
d
as
h
b
o
ar
d

/
a
p
i
/
s
t
a
t
e

y

v
i
s
t
a

w
e
b

Act
ual
iza
r
est
ad
o
en
tie
mp
o
rea
l

Correcto. El
dashboard
mostró
publisher
_online:
true y
cambios en
compartime
ntos
abiertos.

C
o
nt
ro
l
d
es
d
e
el
d

C
o
m
a
n
d
o
s

M
Q

Pe
rmi
tir
sin
cro
niz
aci
ón
y
act
iva

Correcto. El
dashboard
publica en
command/r
equest-
sync y
command/s
et-
monitorin

			r/d
			es
as			act
h			iva
b	T		r
o	T		g.
ar			mo
d			nit
			ore
			o

10.1 Hallazgo técnico importante

Durante las pruebas se encontró que el broker viejo de la BeagleBone Black aceptaba MQTT 3.1, pero no trabajaba bien con clientes configurados en MQTT 3.1.1. Esto causó que el ESP32 lograra abrir el socket TCP, pero se quedara esperando la respuesta MQTT. La solución fue compilar el cliente del ESP32 y el dashboard con protocolo MQTT 3.1.

10.2 Estado observado en la validación

- El broker Mosquitto quedó activo en la BBB.
- El ESP32 se conectó por WiFi y publicó datos al broker.
- El monitor MQTT recibió eventos y heartbeats en tiempo real.
- El dashboard mostró **7 compartimentos abiertos** durante la captura final.

11. Evidencias sugeridas para anexar

- Captura del monitor serie del ESP32 mostrando publicaciones OK.
- Captura de la terminal SSH a la BBB mostrando el puerto 1883 en escucha.
- Captura del monitor MQTT mostrando tráfico real de tópicos.
- Captura del navegador con el dashboard actualizado.
- Video corto donde se vea el cambio de un compartimento y la reacción del sistema.

12. Conclusiones

El proyecto cumplió con la arquitectura pedida en la asignación. Se implementaron de forma correcta los tres roles principales del sistema: publisher, broker y subscriber. El ESP32 envía datos en tiempo real, la BeagleBone Black distribuye los mensajes con Mosquitto y el dashboard reacciona de forma inmediata.

También se comprobó que un problema de compatibilidad de versiones puede afectar la comunicación, aun cuando la red y el puerto estén bien. Al corregir el uso de MQTT 3.1, el sistema quedó funcional. Con esto se demuestra que la solución no sólo funciona, sino que además fue depurada y validada en laboratorio.

13. Anexo de comandos usados en la validación

```
ssh debian@192.168.1.85
```

```
netstat -ltnp | grep :1883
```

```
curl http://127.0.0.1:8080/api/state
```

```
mosquitto_sub -h 192.168.1.85 -t 'escom/iot/equipo7/reed-monitor/#' -v
```

```
PlatformIO: pio run -t upload
```